

«File System Forensics Lab »

LAB#4

Analyse de base en utilisant le Sleuthkit

L'analyse manuelle d'un disque ou d'un système de fichier telle que proposée dans le Lab#3 peut être fastidieuse. Les outils forensics sont là pour vous aider en réalisant pour vous les opérations basiques d'une manière très efficace.

Cependant, il est indispensable de noter que dans certaines situations, les enquêteurs ne peuvent se baser sur ces outils et c'est alors le moment de repasser « en manuel ». Ce que nous verrons dans le Lab#5.

Note : la façon dont le Sleuthkit fonctionne est assez perturbante dans la mesure où quel que soit le système de fichiers analysé, la catégorie « méta-données » fait toujours référence à une notion de *i-noeud*. Cela signifie, par exemple, que dans le cas d'une FAT, la sortie des outils n'affichera pas des valeurs en cluster la plupart du temps. Ne vous en préoccupez pas. La cohérence globale est bien entendu respectée au sein de l'outil.

En utilisant la commande *mmls*, retrouvez les informations obtenues lors de l'analyse des supports *multi1.dd* et *mult2.dd*.

Analysez le fichier *fileadded.dd* en utilisant le programme *fls* et :

- Relevez le numéro i-noeud de *dir101*
- Relevez le numéro i-noeud du programme *testprint*

En utilisant *istat* et les informations obtenues au travers de *fls*,

- Listez les secteurs associés au fichier *testprint*

En utilisant *icat*, extrayez le programme *testprint* sous le nom *testprint.sl* et exécutez-le.

Calculez le sha-256 de cette extraction.

La récupération de fichiers : version simple

Les outils du Sleuthkit peuvent également vous permettre de récupérer facilement des fichiers détruits ou au moins en faciliter l'extraction (on parle alors de « carving »).

Utilisez le fichier *filedeleted.dd*, et essayez, en utilisant un processus semblable à ce que vous aviez fait précédemment, de récupérer le fichier *testprint*. Vous appellerez ce nouveau fichier *testprint.2.sl*.

Comparer le sha-256 du nouveau fichier avec celui calculé précédemment.

Dans le cas idéal, les choses fonctionnent comme vous venez de le faire. Dans la réalité, cela n'est généralement pas aussi évident. D'après vous qu'est-ce qui pourrait ne pas marcher ?

Reprenez votre analyse sur le fichier *filedeleted.mod.dd* du Lab précédent. Conclusion.

La récupération de fichiers : à la dure

Détermination de la technique sur la base d'un fichier connu

Parfois, essayer de récupérer un fichier ou essayer de retrouver des données dans la catégorie « contenue » n'est pas aussi simple, surtout lorsque l'entrée du répertoire a été supprimée ou ré-utilisée.

Dans ce cas, le Sleuthkit peut tout de même vous aider à extraire le fichier.

Si vous cherchez un fichier d'un type donné qui a été supprimé ou pour lequel vous connaissez un « pattern » (séquences d'en-tête ou fin de fichier par exemple), voici comment vous pouvez utiliser le Sleuthkit pour essayer de l'extraire...

- Supposez que vous vouliez récupérer un fichier binaire LINUX. L'en-tête d'un tel fichier contient la chaîne de caractères « ELF ». Vérifiez cela en utilisant *xxd* sur l'un des fichiers *testprint* que vous avez récupérés.
- Puisque vous savez que le fichier a été supprimé, la première approche est donc de vous limiter à rechercher les « data units » marqués comme non-alloués : cela réduira de fait la quantité de données à analyser. L'outil *blkls* permet de faire cette opération. Tapez :

blkls filedelete.dd > unallocated.blk

- Vous pouvez maintenant essayer de localiser la chaîne « ELF » sur le fichier *unallocated.blk*
- Conseil : utilisez le logiciel *hexedit* qui permet de réaliser cette recherche de façon interactive
- Une fois l'offset trouvé dans le fichier des « data units » non-alloués, il vous faut maintenant retrouver le numéro de « data unit » dans le fichier initial. Dans ce but, vous pouvez utiliser la commande *blkcalc*. L'adresse fournie vous donnera l'offset en termes de numéro de secteur.
Note : pour avoir une valeur correcte avec *blkcalc*, indiquez le type de système de fichiers que vous analysez (option -f). L'adresse à fournir à l'outil est le numéro du secteur dans la zone non allouée.
- Indice : vous devriez retrouver l'adresse du premier secteur que vous avez fourni la commande *istat*.
- Vous pouvez maintenant extraire le premier secteur en utilisant la commande *blkcat*. Trouver les autres blocs est cependant plus compliqué car vous devez déterminer ou deviner le nombre de blocs qui étaient initialement associés au fichier en espérant qu'ils soient consécutifs. Dans le cas où vous connaissez la séquence de fin de fichier, vous pouvez bien entendu réutiliser le

mode opératoire précédent.

Note : pour cette première extraction, réalisez le travail en prenant un nombre « complet » de secteurs et sauvegardez le résultat dans le fichier *testprint.carv.v1*

- Calculer le sha-256 du fichier et comparez-le à ceux obtenus précédemment. Pourquoi est-il différent ?

Validez votre hypothèse en utilisant la commande :

diff -y <(xxd testprint.carv.v1) <(xxd testprint.2.sl)

Que pourrait-on faire pour obtenir le fichier original ?

- En utilisant les commandes *blkcat* et *head* (l'option -c peut aider), faites une extraction propre du fichier. Comparez les sha-256 obtenus.

Mise en pratique sur un fichier « perdu »

L'archive mise à disposition recèle un autre fichier dont l'entrée n'est malheureusement plus disponible dans un répertoire. Vous allez essayer de le récupérer en utilisant la méthodologie précédente.

Indications :

- Le fichier en question est un fichier jpg
- Rechercher sur Internet la signature de tels fichiers
- Utiliser la méthodologie précédente avec *blkcalc* pour identifier le premier secteur du fichier
- Explorer les secteurs suivants afin d'identifier une fin possible du fichier.
Indice : le support a été très peu utilisé, la plupart des secteurs sont donc « vides »
- Extrayez le bon nombre de données afin de récupérer le fichier jpg perdu
- Indication : le shasum de votre fichier devrait être :

cbd7f73a8c1b3d37cd3cecc6fc28bae333c753e95aaabf335eb942fc51757f52

D'autres outils forensiques spécialisés dans l'extraction de fichiers existent. Les plus connus sont *foremost* et *scalpel*. L'utilisation de ces outils ne sera pas faite dans le cadre de ce TP.

Annexes

From the "File System Forensic Analysis" by Brian Carrier

The boot sector data structure for FAT12/16 File Systems

Byte Range	Description
0-2	Assembly instruction to jump to boot code
3-10	OEM Name in ASCII
11-12	Bytes per sector
13-13	Sectors per cluster
14-15	Size in sectors of the reserved area
16-16	Number of FATs
17-18	Maximum number of files in the root directory for FAT12/16 - 0 for FAT32
19-20	16-bit value of number of sectors. If the number of sectors is larger than 2-byte value, it is set to 0 and the 4-byte value is found starting at byte 32
21-21	Media type
22-23	16-bit size in sectors of each FAT for FAT12/16 0 for FAT32
24-25	Sectors per track
26-27	Number of heads
28-31	Number of sectors before the start partition
32-35	32-bit value of number of sectors in file system or 0 (if it is defined in byte range 19-20)
36-36	BIOS Int 13h drive number

37-37	Not used	
38-38	Extended boot signature	
39-42	Volume serial Number	
43-53	Volume Label in ASCII	
54-61	File System type Label in ASCII - Informational only	
62-509	Not used	
510-511	Signature value (0xAA55)	

The directory entry in a FAT file system

Byte Range	Description
0-0	First character of file name in ASCII and allocation status (0xe5 if deleted entry or 0x00 if unallocated)
1-10	Characters 2 to 11 of file name in ASCII
11-11	File Attributes (see next table for standard attributes)
12-12	Reserved
13-13	Created time (tenth of seconds)
14-15	Created time (hours, minutes, seconds)
16-17	Created day
18-19	Accessed day
20-21	High 2 bytes of first cluster address (0 for FAT12/16)
22-23	Written time (hours, minutes, seconds)
24-25	Written day
26-27	Low 2 bytes of first cluster address
28-31	Size of file (0 for directories)

Flag attributes in directory entries

Flag value	Description
0x01	Read only
0x02	Hidden file
0x04	System file
0x08	Volume Label
0x0f	LNF
0x10	Directory
0x20	Archive

The LFN structure in a FAT file system

Byte Range	Description
0-0	Sequence Number (ORed with 0x40) and allocation status (0xe5 if unallocated)
1-10	File name characters 1-5 (Unicode)
11-11	File attributes (0x0f)
12-12	Reserved
13-13	Checksum
14-25	File name characters 6-11
26-27	Reserved
28-31	File name characters 12-13